Continuous Patient Vital-Sign Monitoring with Fog-Layer Early Warning: A Fog-to-Cloud Architecture

Sri Venkat Bora
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
Student ID: X25164414
x25164414@student.ncirl.ie

***Abstract*—Clinical deterioration is usually preceded by measurable drift in a patient's vital signs, yet on a busy ward those signs may be recorded only intermittently, so a developing crisis can go unseen between observations. This paper presents a working fog-to-cloud pipeline that monitors patient vital signs continuously and applies early-warning logic at the edge. Each monitored patient carries five instruments, heart rate, blood-oxygen saturation, body temperature, respiration rate, and systolic blood pressure, that are sampled locally and dispatched to a fog gateway at the bedside. The gateway folds each stream into a fixed time window, reduces it to summary statistics, and screens the window against two-sided early-warning thresholds, so that both an abnormally high and an abnormally low value, a tachycardia and a bradycardia, a fever and hypothermia, are flagged in the cycle they occur. Compact per-window aggregates cross a managed message queue into an event-driven ingestion function that stores them; a second, independently deployed function serves a dashboard programming interface behind a managed gateway, and the operator page is delivered as static content from object storage, rendering each patient's heart rate as a live trace. The system is built in Node.js and provisioned to a real Amazon Web Services account by a single infrastructure-as-code apply that created twenty-four resources. Measurement against the live endpoint shows the pipeline healthy end to end within about a minute of start-up, the stored record count rising across polls, and both patients rendering live in a browser, with a hypoxia alert firing on one of them.**

***Index Terms*—fog computing, edge computing, patient monitoring, early warning score, serverless, message queue, Internet of Things.**

I. Introduction

Adverse events in hospital wards, cardiac arrests, unplanned intensive-care admissions, are rarely sudden. They are typically preceded by hours of measurable physiological drift, and structured early-warning scores were developed precisely to catch that drift: by assigning points to how far each vital sign strays from a normal band and summing them, a bedside team can be prompted to escalate before a patient collapses [1]. The National Early Warning Score, now widely standardised, formalises the vital signs to track and the bands that define normality [2]. The limitation these scores share is not the logic but its cadence: a score is only as current as the last set of observations, and on a busy ward those may be taken only every few hours, leaving long intervals in which deterioration is invisible.

Continuous instrumentation removes that blind interval, and connected medical sensing has matured to the point where a patient's vital signs can be streamed and analysed in real time [3]. But the architectural question remains: where should the analysis run? Streaming every raw sample to a central cloud is a poor default. A handful of vital-sign sensors per patient produce readings far faster than any single reading is clinically meaningful, the analysis that matters, is this window abnormal, is simple and local, and a monitoring view that stalls whenever connectivity dips is unacceptable at a bedside. Fog and edge computing address this by moving computation toward the data source [4]. Fog computing inserts a compute tier between the instruments and the cloud, so that windowing, statistical reduction, and early-warning rule evaluation happen at the bedside, while the cloud provides durable storage, elastic query serving, and a globally reachable interface [5], [6]. The cloud services used here follow the standard on-demand, elastic, measured model of cloud computing [7].

This paper reports the design, implementation, and live deployment of such a pipeline. The deployment monitors two patients, each carrying five instruments. The objectives are fourfold. First, to reduce raw samples to compact per-window aggregates at a fog tier at the bedside. Second, to screen each window against two-sided early-warning thresholds at the edge, so that a deviation in either direction is flagged in the cycle it appears. Third, to build a scalable cloud back end that ingests aggregates reliably, stores them durably, and serves them to a live operator dashboard without the write path and the read path contending. Fourth, to deploy the whole system to a real cloud account and verify it end to end. The remainder of the paper describes the architecture and its design decisions, the implementation and its tests, the deployment, an evaluation from the running system, and a concluding reflection.

II. System Architecture and Design

The system is a linear ingestion pipeline with a separate read path for the operator view. Instruments produce readings; a fog gateway at the bedside windows, summarises, and screens them; a message queue carries aggregates to the cloud; an event-driven function persists them; and an independent function reads the stored data back out to a dashboard. Fig. 1 shows the deployed topology. This section takes each tier in turn and then weighs the alternatives.

A. Bedside Sensing and the Fog Gateway

Each patient is represented by five independent sensor processes, one per vital sign: heart rate in beats per minute, blood-oxygen saturation as a percentage, body temperature in degrees Celsius, respiration rate in breaths per minute, and systolic blood pressure in millimetres of mercury. A sensor generates values with a bounded random walk: each new reading steps a small random amount from the previous one and is clamped to a physiologically plausible range, so a stream drifts realistically rather than jumping arbitrarily. Every sensor is governed by two independently configurable intervals, one setting how often a value is produced and the other how often the accumulated values are dispatched to the gateway. Separating sampling from transmission lets a fast-changing vital be sampled often while still being sent to the edge in economical batches. Each dispatch is a small JSON object naming the vital, the patient identifier, and the unit, plus the short list of timestamped values accumulated since the previous dispatch, a payload of a few hundred bytes.

The fog gateway is a small Node.js and Express web service. It accepts posted batches and keeps one running accumulator per combination of vital and patient. Rather than retain raw readings, it folds each incoming value immediately into a compact record holding the count, running sum, minimum, maximum, and most recent value, so its memory footprint stays flat regardless of how many readings arrive. On a fixed cadence it closes the current window, sealing each accumulator into a summary carrying the window bounds, the count, the minimum, the maximum, the mean, and the latest value, and screening that summary against the early-warning thresholds for its vital before forwarding it.

B. Edge Early-Warning Rules

Unlike a fault that only manifests in one direction, a vital sign can be dangerous when it is too high or too low, so most vitals here are screened against a two-sided threshold. Table I lists the rules and the clinical condition each encodes. Nine conditions are evaluated across the five vitals: bradycardia and tachycardia on heart rate, hypoxia on oxygen saturation, fever and hypothermia on temperature, bradypnea and respiratory distress on respiration, and hypotension and hypertension on blood pressure. Most rules test the window mean, because a sustained average is a more reliable signal than one noisy sample; the hypothermia rule instead tests the window minimum, because the coldest reading in the window is the safety-relevant one. Screening at the edge means a deviation is flagged in the same cycle as the readings that reveal it, without a round trip to the cloud, so the early-warning logic keeps pace with the data.

TABLE I
Edge-Evaluated Early-Warning Rules

Vital	Rule (per window)	Alert raised
Heart rate	mean below 50 / above 120 bpm	Bradycardia / tachycardia
Oxygen saturation	mean below 92 %	Hypoxia
Body temperature	mean above 38.5 °C; min below 35.5 °C	Fever; hypothermia
Respiration rate	mean below 10 / above 24	Bradypnea / respiratory distress
Systolic pressure	mean below 90 / above 140	Hypotension / hypertension

When a window closes the gateway does not send one message per patient per vital. It collects every summary produced in that cycle and hands the whole set to a single batched send toward the queue, splitting the set into chunks only where the queue interface caps the number of entries in one batched call. For a two-patient deployment producing up to ten summaries per cycle, this turns ten network calls to the cloud into one, lowering request cost and per-message overhead.

C. Queue-Decoupled Ingestion and the Reading Store

Aggregates leave the fog tier through a managed message queue rather than a direct call into the storage function. The queue is a buffer and a shock absorber: it holds messages durably until consumed, so a slow or briefly unavailable consumer neither loses data nor pushes back on the gateway, and a burst of aggregates is smoothed to a rate the consumer can sustain, the decoupling that message-oriented middleware provides between producers and consumers [8]. The decoupling covers the consumer side of the queue, not the hop onto it: if the gateway's batched send fails, the window has already been sealed and cleared, so the gateway logs the failure and moves on rather than retrying or buffering locally, a stated trade-off accepted at the data volume a two-patient deployment produces.

An event-driven ingestion function consumes the queue, transforms each aggregate into a stored item, and writes it to a managed key-value store. The store uses a two-part key. The partition key is the vital, which groups all readings for a given sign and makes the dominant query, "give me the recent windows for this vital," a single efficient lookup. The sort key concatenates the window-end timestamp with the patient identifier. The timestamp gives a natural chronological order within each partition, and appending the patient identifier is deliberate: when both patients close a window for the same vital in the same cycle, their two items share a partition key and a window-end time, and without the patient identifier in the sort key the second write would overwrite the first.

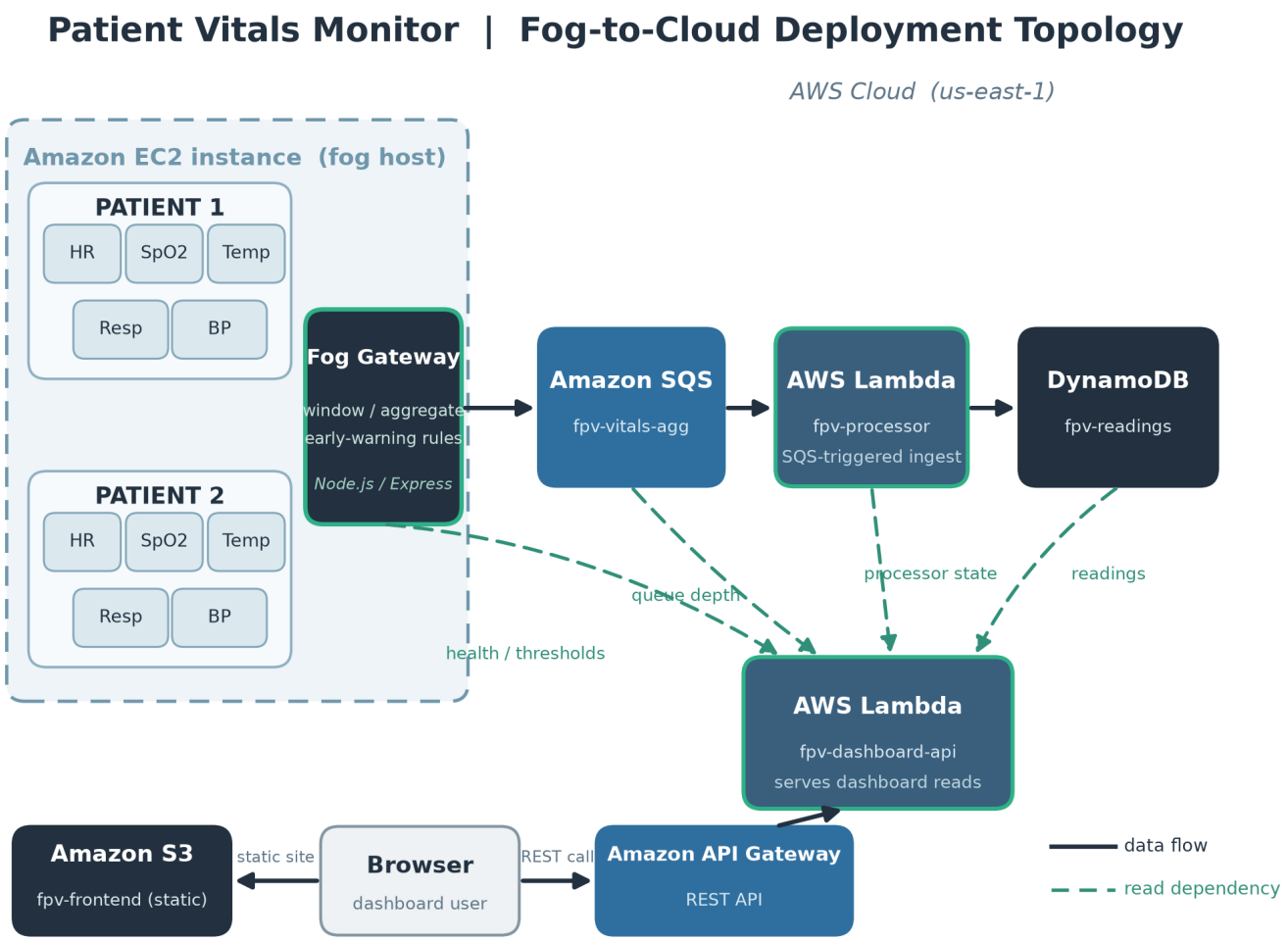

Fig. 1. Fog-to-cloud deployment topology. The fog gateway and the ten sensor processes run in containers on a single edge host at the bedside. Solid arrows are the ingestion data flow from the instruments through the queue and the ingestion function into the store. Dashed arrows are the read dependencies the dashboard function assembles from the gateway, the queue, the ingestion function, and the store. The browser loads static assets from object storage and calls the dashboard programming interface through the managed gateway.

Concatenating the identifier keeps both patients' items distinct in one partition.

D. Read Tier: Interface, Live Trace, and Ward View

The operator view is served by a second cloud function deployed independently of the ingestion function, so the read path and the write path scale and fail separately. This function answers a small read-only programming interface: a patients endpoint returning, for every patient, the latest reading and window statistics on each vital; a readings endpoint returning a recent series for one vital and patient, used to draw the live heart-rate trace; a health endpoint; a back-end statistics endpoint; and a thresholds endpoint that proxies the gateway's published rules. The health endpoint reports four independent booleans, that the gateway is reachable, the queue is reachable, the ingestion function is active, and the pipeline is flowing, meaning the freshest stored window is recent, together with the age of the freshest window.

The static frontend, a single page, a stylesheet, the page script, and a locally vendored charting library, is served from object storage. The page renders each patient as a card whose primary element is a live trace of the heart rate drawn from the readings endpoint, presented in the manner of a bedside monitor, with the other four vitals shown as tiles carrying their latest value, window range, and a flag when a threshold is breached. A ward summary counts the patients monitored and the vitals currently out of range, and a critical banner names any patient with a firing alert. The page script re-fetches the patients view, the health endpoint, and the back-end statistics together on a short interval and repaints from each response. The frontend learns which programming-interface origin to call from a single inline value in the page that is substituted with the real gateway origin at upload time; in local development, where one process serves both the interface and the page, the value is left unset and calls default to the same origin.

E. Alternatives Considered

Each choice below was made for a stated reason.

A queue rather than a direct function call. The gateway could invoke the storage function directly and synchronously, removing the queue and one hop of latency. This was rejected: a synchronous call couples the edge to the availability and speed of the cloud function, so if the function is throttled, cold, or unavailable the gateway blocks or drops data and back pressure reaches the instruments. The queue breaks that coupling, absorbs bursts, retries delivery on the consumer's behalf, and lets the gateway consider an aggregate handed off the instant it is enqueued [8]. The extra latency is irrelevant on a window cadence; the durability and decoupling are what a bedside monitor's uplink needs.

A key-value store rather than a relational database. The access pattern is narrow and known in advance: append time-ordered aggregates, then read back the recent windows for a vital. A key-value store built for that pattern of high-volume writes and key-based reads gives predictable performance at scale without the operational weight of a relational engine, the trade-off the original Dynamo design articulated and the managed service inherits [9], [10]. The chosen key scheme turns the query the dashboard needs into a single partition lookup.

Event-driven functions rather than an always-on server. The ingestion and interface tiers run as event-driven functions, which suits a bursty and, for the interface, intermittent workload: the account pays per request rather than for idle capacity, and the platform scales the functions as load varies. The known cost is cold-start latency after a function has been idle [11]; here the effect is minor, because a steady stream of queue messages keeps the ingestion function warm and the dashboard's short polling interval does the same for the interface.

F. Security and Privacy Posture

The edge host exposes a single inbound port carrying only the gateway's health and threshold endpoints; it has no remote-login key pair and is administered through the cloud provider's systems-management channel, removing the most common remote-access attack surface. Both cloud functions run under the account's shared laboratory role, because the account cannot create new roles; the stated production intent is to split this into least-privilege roles, granting the ingestion function only the ability to receive and delete queue messages and write items, and the dashboard function only read-oriented permissions. The telemetry in this deployment is simulated and carries no patient identity, only a synthetic patient label, so no personal or protected health information is stored or served; in a real clinical setting the same interface would additionally require authentication, encryption in transit and at rest, and access control, because vital-sign data is sensitive. Because the interface is served from one origin and called from another, every response, including error responses, carries an explicit cross-origin header so a browser accepts the reply under the rules that govern cross-origin requests [12].

TABLE II
AUTOMATED TEST COVERAGE BY MODULE

Module	Tests
Sensors	4
Fog gateway	16
Ingestion function	5
Dashboard interface	16
Total	**41**

III. IMPLEMENTATION

The entire stack is implemented in Node.js with no build step and no TypeScript. The fog gateway is built on the Express web framework; the cloud clients for the queue, the store, and the functions come from the official provider software development kit for JavaScript. The dashboard's heart-rate trace is drawn with a charting library vendored into the project and served locally rather than from a content delivery network, so the page carries no third-party runtime dependency. For local development and continuous integration the full stack runs under Docker Compose against a local emulator of the cloud services, so the same code runs unchanged against the emulator or the real account by changing only endpoint and credential configuration.

A. Dashboard Handler and the Runtime Argument Guard

Deploying the dashboard interface as a cloud function raised two implementation points worth noting. First, its request routing is expressed as a small ordered route table: each entry pairs a path with a handler, and the incoming request is matched against the table, which keeps the dispatch declarative and easy to extend with a new endpoint. Second, to keep the handler testable it accepts an optional argument carrying pre-constructed cloud clients, which the unit tests supply as fakes. The serverless runtime, however, invokes a handler with a completion callback as one of its arguments, so the handler admits that argument as injected clients only when it actually carries a store client, and otherwise builds its own; the fake-client path the tests rely on and the self-constructing path production needs are thereby distinguished by the shape of the argument rather than merely its presence, which prevents the runtime's callback from being mistaken for injected clients.

B. Automated Test Suite

Testing uses the test runner built into the Node.js runtime, with no external framework. Cloud-facing code accepts an injected client, so the unit tests exercise it against small hand-written fake clients rather than a live service or even the emulator, keeping the suite fast, deterministic, and free of network dependence. The suite totals 41 tests across the four modules, distributed as shown in Table II, with the dashboard total including tests that assert the runtime argument guard described above.

C. Continuous Integration

A continuous-integration workflow runs on every push and every pull request as two dependent jobs. The first installs dependencies and runs the unit suites for all four Node.js modules. Only if that job passes does the second run: it brings up the emulator-backed Docker Compose stack, executes an end-to-end check that pushes data through the sensors, the gateway, the queue, the ingestion function, and the store, and then tears the stack down. Gating the integration job on the unit job keeps feedback fast in the common case, where a logic error is caught by the quick unit suite, while still exercising the assembled system before a change is accepted.

D. Infrastructure-as-Code Deployment

The system was deployed live to a real provider laboratory account in the North Virginia region. Provisioning is expressed as infrastructure as code and applied in an isolated workspace so that it plans only its own resources: a single apply created twenty-four resources with no manual command-line steps, the plan reporting twenty-four to add and none to change or destroy. The live resources comprise a key-value table, a message queue, the ingestion function and the dashboard function reached through a REST programming-interface gateway, an edge compute instance whose firewall admits only the single gateway port and which carries no login key pair, a fixed public address attached to that instance, and two object-storage buckets, one serving the dashboard frontend and one used for deployment staging. The application code and the infrastructure-as-code configuration are documented in the repository at [GitHub repository URL].

The system was then evaluated by measuring the running deployment, not by inspecting code alone; all figures below were read from the live endpoint.

IV. Live Evaluation

A. End-to-End Liveness

Within about a minute of the edge instance finishing container start-up, the health endpoint reported all four indicators true, the gateway reachable, the queue reachable, the ingestion function active, and the pipeline flowing, and the freshest stored window was a few seconds old. A freshest-window age in single-digit seconds means a reading produced at the bedside had already traversed the sensor, the gateway, the queue, the ingestion function, and the store and was being read back by the dashboard, all within seconds, the property that distinguishes a live pipeline from a set of separately deployed components.

B. Accumulation and a Firing Alert

The stored record count was polled repeatedly across the verification window and was observed to climb across successive polls, confirming that the ingestion function was consuming the queue and writing continuously. Over the same window the patients endpoint returned live per-patient data for both patients, five current vitals each with their window range and mean. During verification one patient's oxygen saturation drifted below the hypoxia threshold, and the pipeline behaved end to end as designed: the edge rule raised a hypoxia alert on the window, the alert propagated through the queue and the store to the read tier, the ward summary counted one vital out of range, and the dashboard's critical banner named the patient and the hypoxia condition while the patient's oxygen-saturation tile was flagged. All static frontend assets were fetched successfully from object storage, the cross-origin header was present on the interface responses, and the inline interface origin in the page had been correctly substituted with the real gateway address at upload time.

Fig. 2 is a screenshot of the deployed dashboard rendered in a real browser. It shows both patient cards with their live heart-rate traces and vital tiles, the ward summary reporting one vital out of range, and the critical banner naming the hypoxia alert on the affected patient. The screenshot is evidence that the system renders correct, live, clinically annotated content to an operator, not merely raw numbers.

C. Cost

The design keeps standing cost low. Every cloud service in the pipeline except the edge instance is billed per request and, at the volumes this deployment produces, stays within the provider's free tier: the queue, the two functions, the key-value store, the interface gateway, and the object storage all bill on use. The edge instance is the only continuously billed resource, and because the serverless dashboard and interface do not depend on it being up, only fresh readings and the gateway's own health reading do, it can be stopped while idle without taking the dashboard offline. The standing cost when idle is therefore essentially that of one small instance, and even that can be paused.

D. Limitations

Three limitations are stated plainly. First, the sensor and fog tier runs on a single edge instance, a single point of failure for that tier: if the instance stops, new readings stop, though previously stored data and the serverless read path remain available. A production bedside deployment would replicate the fog tier or run a gateway per bay. Second, the laboratory account is session based, so only single-session behaviour was verified; multi-day stability and credential rotation over long runs were not exercised. Third, both cloud functions currently share one broadly scoped account role; splitting them into least-privilege roles is specified but was not possible in an account that cannot create new roles.

V. Conclusion

This work set out to move patient vital-sign monitoring from intermittent observation toward continuous, edge-screened surveillance, and to do so with a distributed design that places computation where it belongs: windowing, statistical reduction, and early-warning rule evaluation at a fog tier at the bedside, durable storage and elastic serving in the cloud. The result is a working pipeline that monitors two patients and

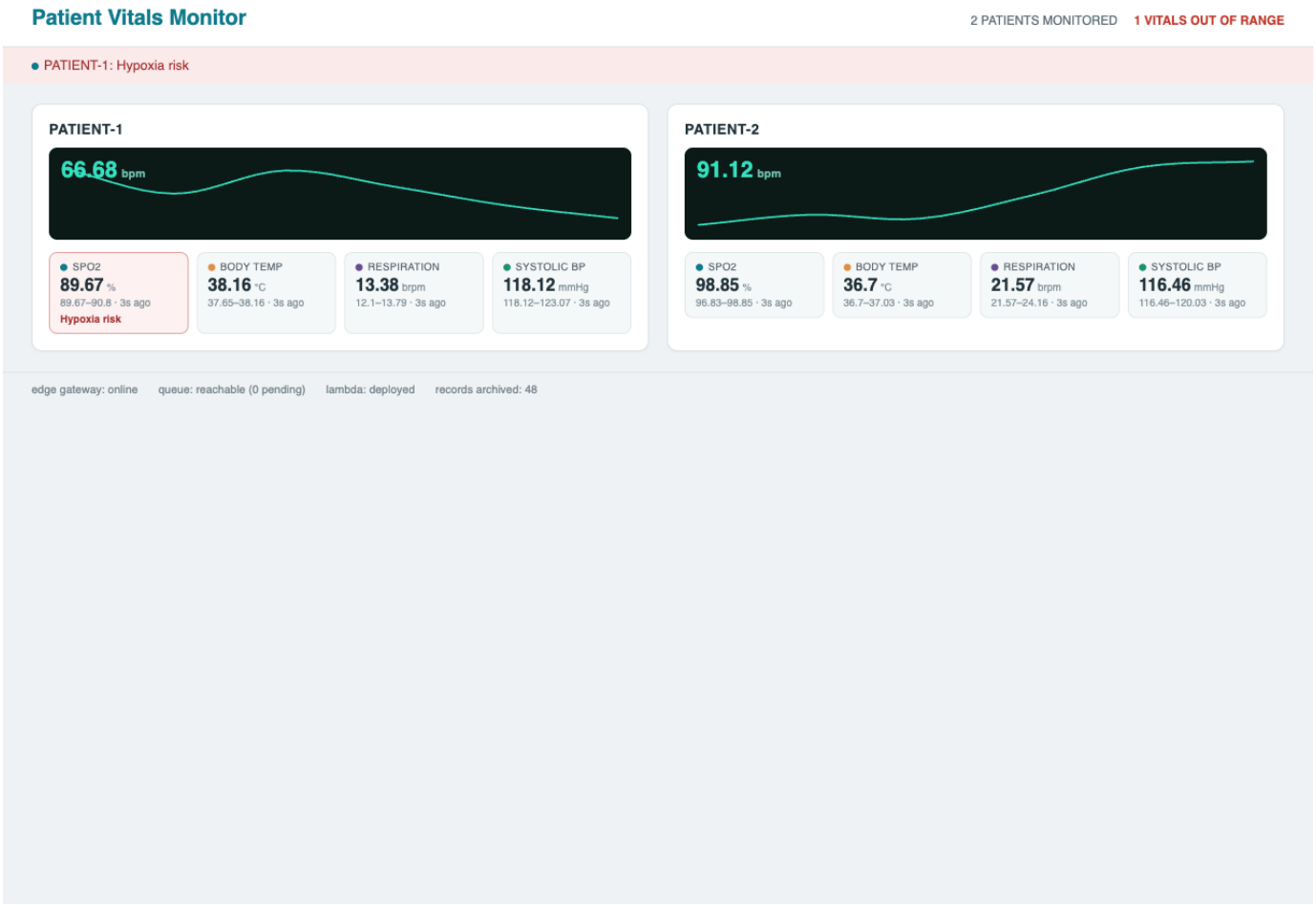

Fig. 2. The deployed operator dashboard rendered live in a browser. Each patient card shows a live heart-rate trace and the other four vitals as tiles with their window range; the header reports two patients monitored and one vital out of range; the banner names the firing hypoxia alert, and the affected oxygen-saturation tile is flagged.

five vitals each, screens every vital against a two-sided early-warning threshold at the edge, and presents a live bedside-style dashboard drawn from a real cloud deployment.

Several lessons stand out. The value of the message queue was not latency but resilience: decoupling the bedside from the cloud consumer is what makes the pipeline tolerant of a lossy uplink. The sort-key design was a reminder that a small modelling decision, appending the patient identifier to a timestamp, is the difference between two patients coexisting and one silently overwriting the other. A further lesson concerned the boundary between test seams and the runtime: a handler that accepts injected clients for testing must distinguish them from the arguments the serverless runtime passes of its own accord, or it will fail in production while passing every local test. Live verification confirmed the design end to end when a real hypoxia excursion travelled from an edge rule to a named alert on the operator's screen. Natural next steps are replicating the fog tier for high availability, splitting the shared role into least-privilege roles, adding authentication and encryption for a real clinical setting, and extending the edge rules from fixed thresholds toward the additive scoring that formal early-warning systems use, so that several mild deviations together raise concern before any single one crosses its line.

References

[1] C. P. Subbe, M. Kruger, P. Rutherford, and L. Gemmel, "Validation of a modified early warning score in medical admissions," *QJM: An International Journal of Medicine*, vol. 94, no. 10, pp. 521–526, 2001.

[2] Royal College of Physicians, "National early warning score (news) 2: Standardising the assessment of acute-illness severity in the NHS," Royal College of Physicians, London, Tech. Rep., 2017.

[3] S. M. R. Islam, D. Kwak, M. H. Kabir, M. Hossain, and K.-S. Kwak, "The internet of things for health care: A comprehensive survey," *IEEE Access*, vol. 3, pp. 678–708, 2015.

[4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. 1st Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12)*, 2012, pp. 13–16.

[6] R. K. Naha, S. Garg, D. Georgakopoulos, P. P. Jayaraman, L. Gao, Y. Xiang, and R. Ranjan, "Fog computing: Survey of trends, architectures, requirements, and research directions," *IEEE Access*, vol. 6, pp. 47 980–48 009, 2018.

[7] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-145, 2011.
[8] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
[9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, 2007, pp. 205–220.
[10] M. Elhemali, N. Gallagher, N. Gordon, J. Idziorek, R. Krog, C. Lazier, E. Mo, A. Mritunjai, S. Perianayagam, T. Rath, S. Sivasubramanian, J. C. Sorenson, S. Sosothikul, D. Terry, and A. Vig, "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '22)*, 2022, pp. 1037–1048.
[11] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '18)*, 2018, pp. 133–146.
[12] A. Barth, "The web origin concept," Internet Engineering Task Force, Tech. Rep. RFC 6454, 2011.